Does INT8 make Leela stronger per second?

Quantizing the BT3 transformer in Leela Chess Zero with TensorRT, and measuring how much of the error the search absorbs



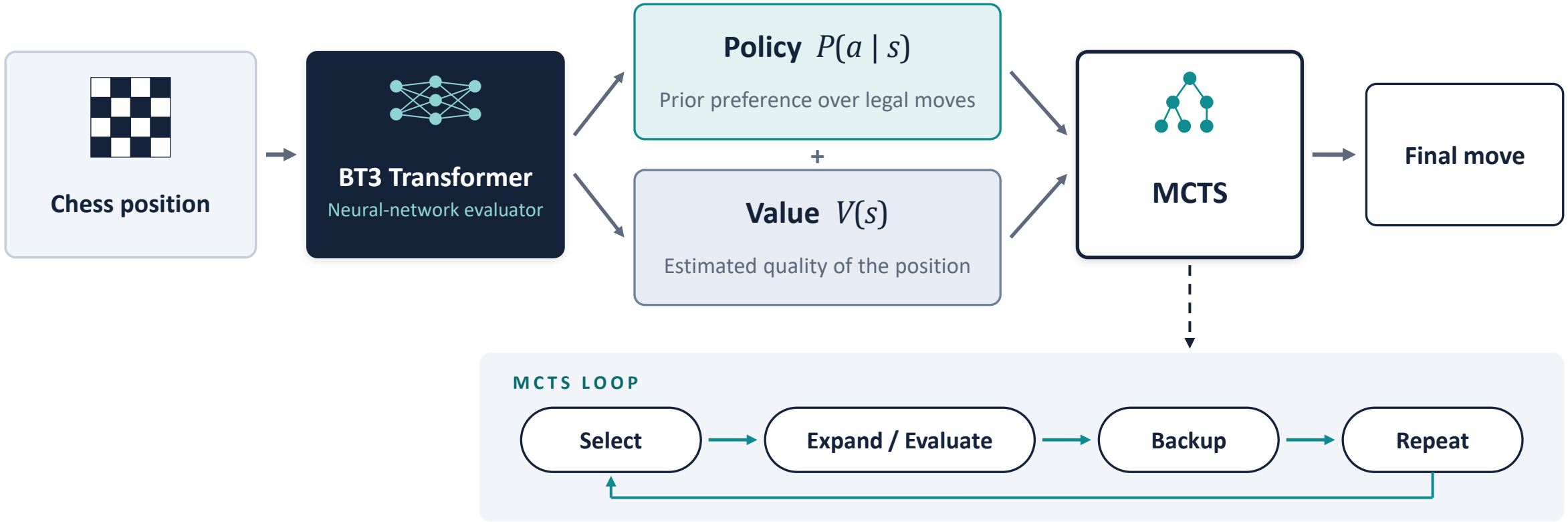
CORE QUESTION

At the same wall-clock time, does the extra search from faster INT8 inference make up for the network quality it loses?

Ritwik Mishra · ME23B237
Anjan R D · ME23B157

CS6886 · Systems for Deep Learning
IIT Madras

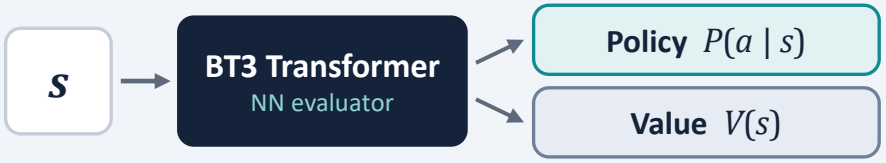
How Leela Chess Zero works



The network guides the search. The final move comes from the search statistics, not straight from the network.

How MCTS uses policy and value

AT STATE s



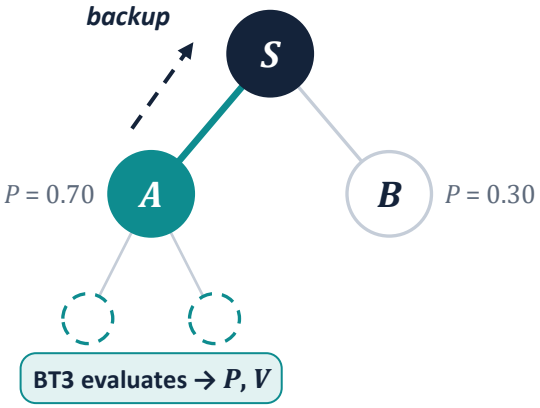
PUCT SELECTION RULE

$$a^* = \text{argmax}_a \left[Q(s,a) + c_{puct} P(s,a) \frac{\sqrt{\left(\Sigma_b N(s,b) \right)}}{1 + N(s,a)} \right]$$

P = neural-network prior · **Q** = average backed-up branch value · **N** = visit count

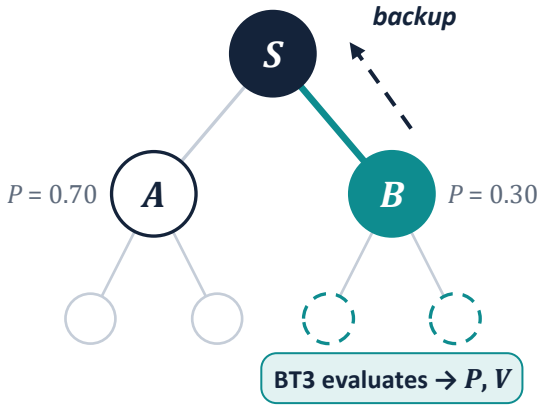
SIMULATION 1

Select **A** → new leaf → BT3 evaluates it (P, V) → expand → back up



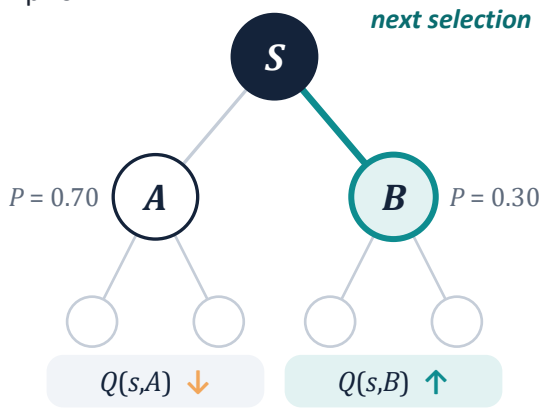
SIMULATION 2

Exploration term lets **B** be selected → BT3 evaluates **B** → expand → back up

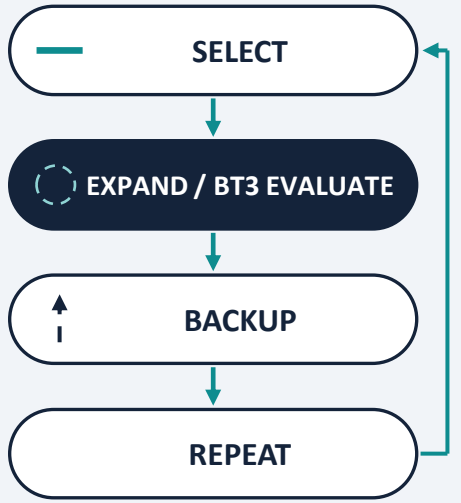


SIMULATION 3

Updated Q and N may make **B** more attractive, although **A** had the higher prior



MCTS LOOP



INT8 adds error to both P and V. The search can correct that error, or it can amplify it.

Why network accuracy alone isn't enough

A different top move is not always a worse move

TABLEBASE EXAMPLE

FP16 model *prefers* Move A

INT8 model *prefers* Move B

CASE 1

Move A	Win
Move B	Win

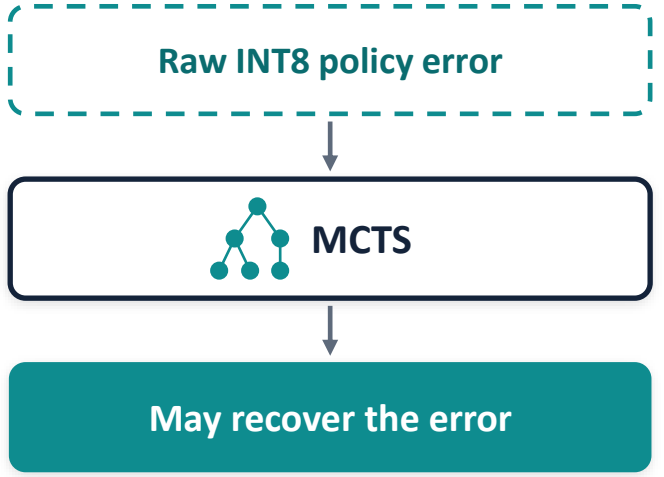
The top move changed, but both moves still win.

CASE 2

Move A	Win
Move B	Draw

INT8 turned a win into a draw. That is a real error.

Leela is a network plus a search



A different network output doesn't always mean a worse final move.

So we measure both how much the network changes and whether the final move is still correct.

Prior evidence and our hypothesis

PRIOR WORK

Haque, Wei and Müller, ACG 2021

"On the Road to Perfection? Evaluating Leela Chess Zero Against Endgame Tablebases"

- They evaluated raw Lc0 policy decisions using exact endgame tablebases.
- They then evaluated the same positions after MCTS search.
- Search corrected almost all of the raw network errors.

KQkq tablebase positions · Strong network · 2,055,004 positions tested

3075 Raw policy mistakes

400 MCTS simulations →

36 Mistakes after search

Note: *the study used an older Lc0 network. We cite it as evidence that search corrects network errors, not as evidence about BT3 or INT8.*

OUR HYPOTHESIS

INT8 → Network quality ↓

BUT

INT8 → Inference throughput ↑ → More MCTS search in the same time

HYPOTHESIS

An INT8 BT3 may be weaker per evaluation, but stronger per second if the faster inference buys enough extra search.

Haque et al. found that search fixes most raw network errors. We test whether it also fixes the errors INT8 adds.

Three questions, from the network to the whole engine

1

RQ1

Network damage

How much does INT8 change BT3's policy and value outputs, and which compression techniques shrink that change?

Measured by
Policy KL, top-move agreement, value error, raw-policy tablebase accuracy

2

RQ2

What search hides

With the same number of search nodes, how much of the damage does MCTS correct? Does it cope better with policy error or with value error?

Measured by
Tablebase accuracy at fixed node budgets, and the policy/value swap test

3

RQ3

Does speed pay off?

With the same wall-clock time, or the same energy, does faster inference buy enough extra search to make up for the lost quality?

Measured by
nodes/s, accuracy at equal time and equal energy, INT8 vs FP16 matches

WHAT WE COMPARE

FP16
The baseline. BT3 in TensorRT at half precision

→

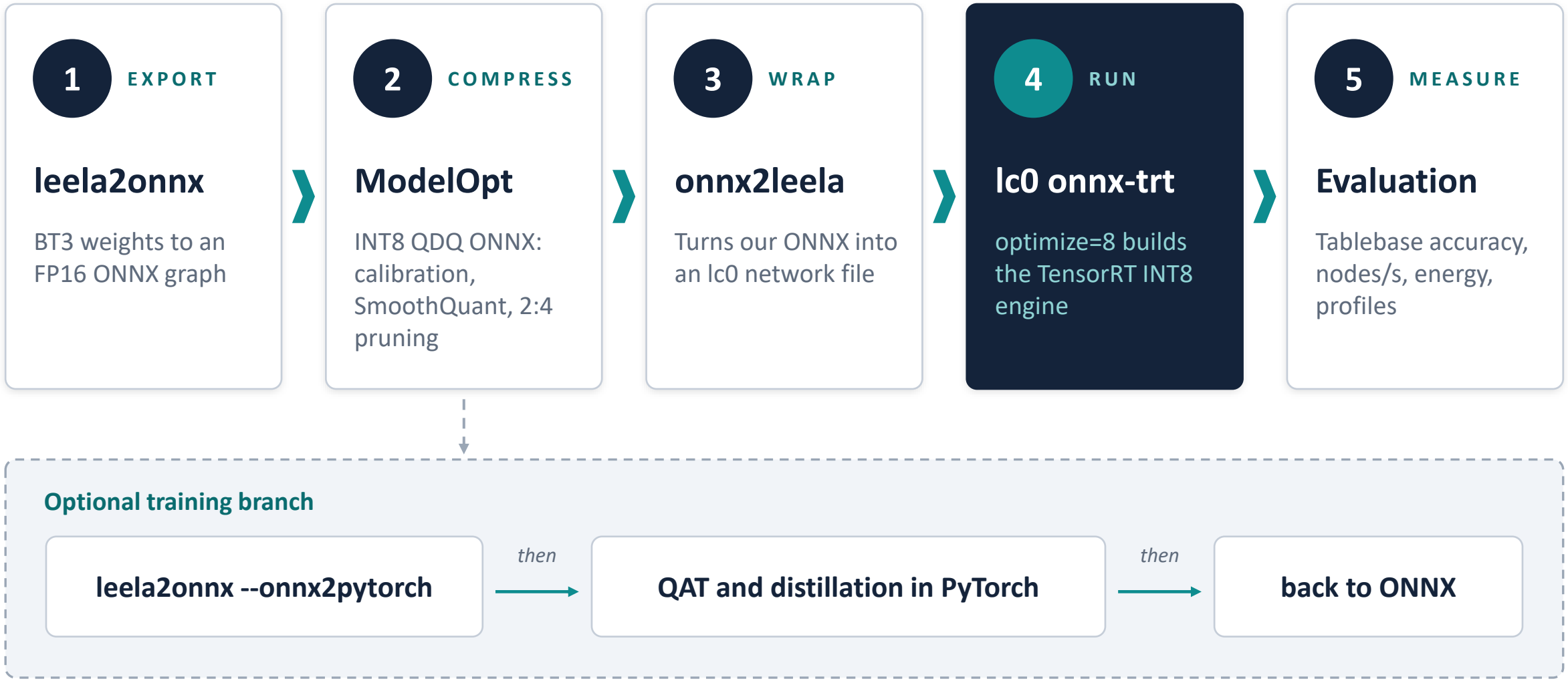
INT8 (W8A8)
Weights and activations in 8 bits, run on the INT8 tensor cores

→

2:4 sparse INT8
INT8 plus pruning in the one pattern the tensor cores can skip

Every model runs inside the real lc0 engine through TensorRT. FP8, W4 and custom kernels are out of scope.

From Leela network to measured result



Compression ablation: one technique at a time

MAIN LADDER: EACH STEP KEEPS EVERYTHING ABOVE IT

0	**FP16**	Baseline	
1	**Naive W8A8 PTQ**	Per-tensor weights, max calibration, random positions	Must
2	**+ per-channel weights**	A separate scale for each output channel	Must
3	**+ better calibration**	Entropy or percentile instead of max	Must
4	**+ SmoothQuant**	Moves activation outliers into the weights	Must
5	**+ search-aware calibration**	Calibrate on positions the search actually visits	Must
6	**+ QAT**	Fine-tune with fake quantization	Should
7	**+ distillation**	Train against FP16 outputs during QAT	Should

SPARSE BRANCH

Prune the FP16 weights first, then quantize with the best recipe from the ladder

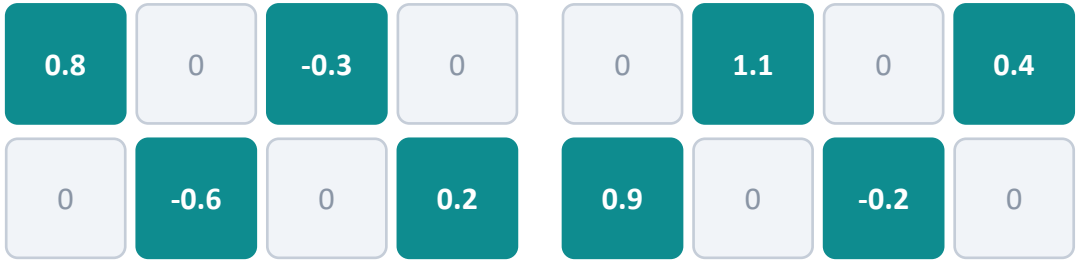
S0	**Magnitude 2:4 + INT8** Speed ceiling, no data needed	Must
S1	**SparseGPT 2:4 + INT8** Keeps accuracy without training?	Should
S2	**SparseGPT 2:4 + INT8 + KD** Recovery training, only if QAT runs	Could

Why the split matters

Steps 1 to 7 all run at the same speed, so they are a quality study. Sparsity changes speed, so the best sparse model also goes into the timed studies.

2:4 sparsity: pruning in the pattern the GPU can skip

THE PATTERN



Two weight rows, split into groups of four

- In every group of four weights, exactly two are zero.
- The sparse tensor cores on the RTX 4060 skip the zeros, which halves the multiplications on paper.
- End to end we expect far less than 2x, because only the matmuls get faster.

WHICH TWO WEIGHTS GO?

	Magnitude 2:4	SparseGPT 2:4
Rule	Drop the two smallest	Drop the two whose loss changes the output least
Data	None	Calibration positions
Repair	None	Adjusts the surviving weights
Our use	Speed ceiling (S0)	Accuracy without training (S1)

Both leave the same pattern, so both run at the same speed. SparseGPT reuses the Hessian code from our Assignment 2 GPTQ implementation.

ORDER OF OPERATIONS

FP16 weights → **Prune to 2:4** → **INT8, calibrated on the pruned net** → **TensorRT build with sparsity on** → **Check layer info for sparse kernels**

Three comparisons against FP16

A · Network only

QUESTION

How much does each model change the network?

MODELS

Every ablation step

MEASURE

- Policy KL against FP16
- Top-move agreement
- Value/WDL error
- Raw-policy tablebase accuracy

B · Fixed nodes

QUESTION

How much of the damage does search hide?

MODELS

Every ablation step

MEASURE

- Tablebase accuracy at a fixed node budget
- Node-equivalents: extra nodes FP16 needs to match each model

Node budgets: 0 · 100 · 400 · 800 · 1600

C · Fixed time

QUESTION

Does the speed pay off?

MODELS

FP16, best INT8, best 2:4

MEASURE

- nodes/s in lc0
- Tablebase accuracy at equal wall-clock time
- About 1000 games of INT8 vs FP16 at 100 ms per move

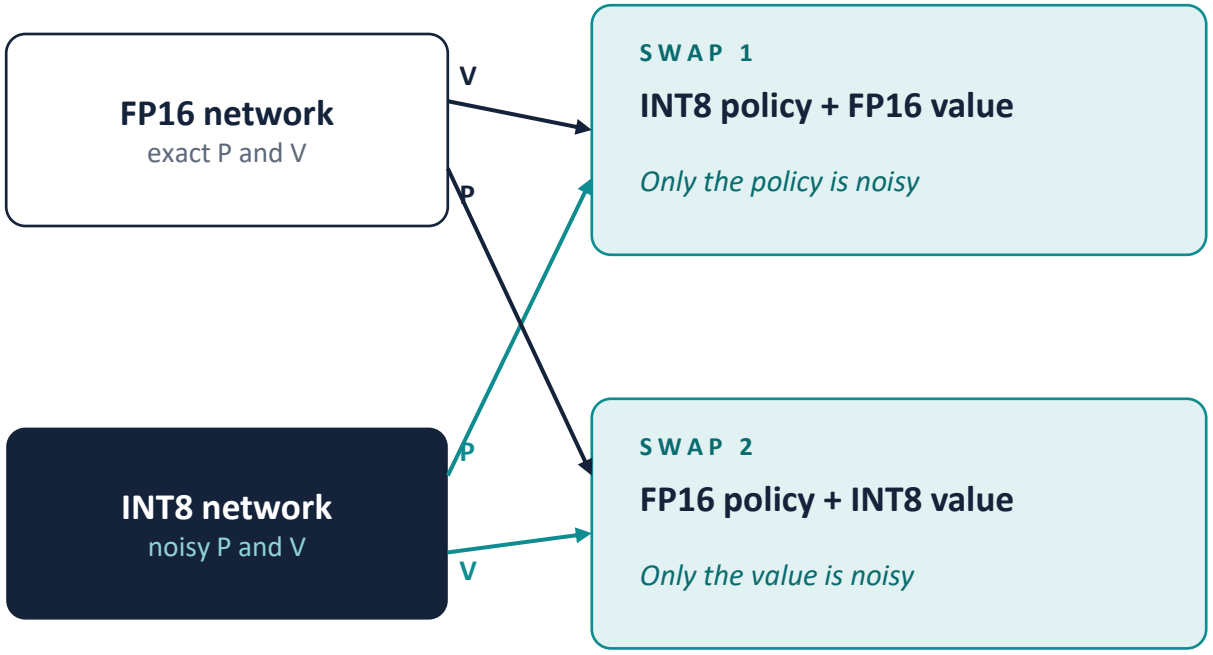
HOW A MOVE IS SCORED

Wrong if it changes the tablebase result. DTZ-optimal rate is a secondary number.

Win → Draw or Loss *or* **Draw → Loss**

Test positions: Syzygy endgames, plus a hard subset where FP16 is close to getting the move wrong.

Does search cope better with policy error or value error?



Both mixes go into the same fixed-node search and are scored against FP16 and full INT8. Two networks per position make timing meaningless, so this runs at fixed nodes only.

WHY NOT KEEP ONE HEAD IN FP16?

Almost all of BT3's compute is in the shared body. The policy and value heads are small layers on top of it. Keeping one head in FP16 still leaves it reading noisy INT8 features, so both variants would look like full INT8. Swapping the outputs of two whole networks isolates each error cleanly.

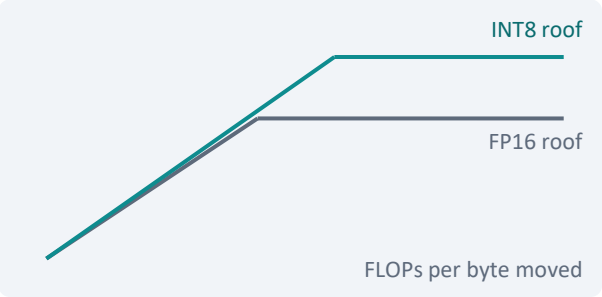
HOW WE READ THE RESULT

- If swap 1 stays close to FP16 and swap 2 drops, the search absorbs policy error but not value error.
- A plausible reason: search explores past a bad policy, but a wrong value is backed up the tree. We will test this, not assume it.

This tells us which part of the INT8 error the search can repair.

Hardware studies that explain the speed numbers

1 Profile and roofline

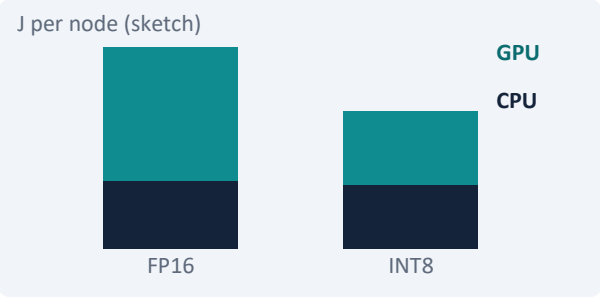


HOW

Time per layer from trtexec. Each matmul placed against the 4060's compute and memory limits.

Shows which layers INT8 speeds up and which ones set the limit.

2 Energy per node

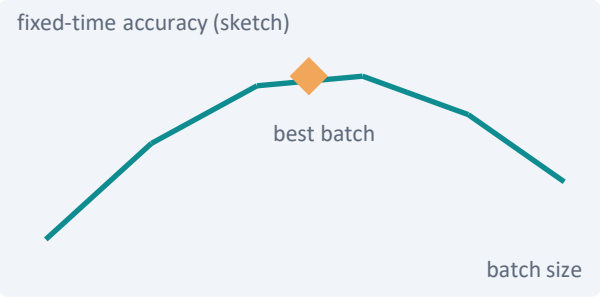


HOW

GPU energy from the NVML counter, CPU energy from RAPL, both above idle. Accuracy at equal energy budgets.

Tests whether INT8 is stronger per joule as well as per second.

3 Search batch size



HOW

nodes/s and tablebase accuracy for lc0 minibatch sizes from 8 to 512.

Big batches use the GPU better but make search less exact. The best size may differ for FP16 and INT8.

Timed runs use a locked GPU clock where the laptop allows it, warm-ups, repeated runs and medians. Sketches show the plot layout only, not results.

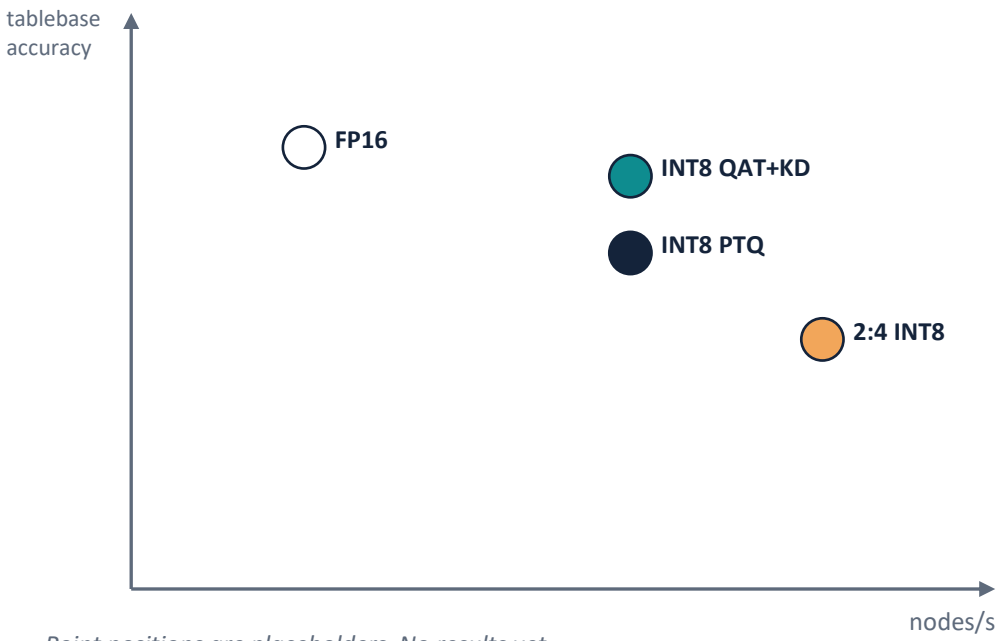
Four weeks, two people, four gates

	Week 1 FP16 end to end	**Week 2** INT8 ladder	**Week 3** Training and timing	**Week 4** Analysis and report
PERSON A model	• Export BT3, build lc0 onnx-trt • Tablebase test set and splits • Outlier check	• Ablation steps 1 to 5 • Studies A and B on every step • Mid-week: QAT go or drop	• QAT and KD (if kept) • SparseGPT (S1) • Policy/value swap study	• Ablation and swap plots • Method and results sections
PERSON B systems	• Day-1 test: lc0 runs our INT8 file • FP16 baselines: nodes/s, profile, energy • Clock-lock check	• An INT8 engine per step • Check matmuls really run in INT8 • 2:4 speed ceiling (S0)	• Study C and overnight matches • Energy and batch-size sweeps • Roofline	• Timing, energy and roofline plots • Check every number against logs
GATE	C1 **FP16 runs end to end**	C2 **Best PTQ frozen: safety net**	C3 **All results frozen**	C4 **Report and code frozen**

After gate C2 the project is complete even if nothing else works. QAT, SparseGPT and sparse recovery are extras on top of it.

What we will hand in

HEADLINE FIGURE (F11), LAYOUT ONLY



Point positions are placeholders. No results yet.

Code	Export, quantization ladder, SmoothQuant, sparsity, training, swap MCTS, benchmarks. One script per experiment, reproducible from a fresh clone.
Models and data	ONNX files, lc0 network files, engine caches and calibration data for every model. Tablebase test set, hard subset and splits.
Tables	T1 ablation · T2 speed · T3 energy · T4 matches · T5 2:4 speed ceiling
Figures	F1 ablation ladder · F2 accuracy vs nodes · F3 node-equivalents · F4 policy/value swap · F5 accuracy vs time · F6 profile and roofline · F7 J/node · F8 equal-energy accuracy · F9, F10 batch size · F11 frontier · F12 outliers
Report and demo	Full write-up, plus the INT8 engine playing in lc0 with nodes/s on screen.

Success means an honest accuracy-per-second answer for INT8 on BT3, whichever way it comes out.

References

[1] **Haque, R., Wei, T.-H., Müller, M.** On the Road to Perfection? Evaluating Leela Chess Zero Against Endgame Tablebases. Advances in Computer Games, 2021.

[2] **Frantar, E., Ashkboos, S., Hoefler, T., Alistarh, D.** GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers. ICLR, 2023.

[3] **Xiao, G., Lin, J., Seznec, M., Wu, H., Demouth, J., Han, S.** SmoothQuant: Accurate and Efficient Post-Training Quantization for Large Language Models. ICML, 2023.

[4] **Frantar, E., Alistarh, D.** SparseGPT: Massive Language Models Can Be Accurately Pruned in One-Shot. ICML, 2023.

[5] **Mishra, A. et al.** Accelerating Sparse Deep Neural Networks. arXiv:2104.08378, 2021.

[6] **Jacob, B. et al.** Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference. CVPR, 2018.

[7] **Hinton, G., Vinyals, O., Dean, J.** Distilling the Knowledge in a Neural Network. arXiv:1503.02531, 2015.

[8] **Williams, S., Waterman, A., Patterson, D.** Roofline: An Insightful Visual Performance Model for Multicore Architectures. Communications of the ACM, 2009.

[9] **Silver, D. et al.** Mastering Chess and Shogi by Self-Play with a General Reinforcement Learning Algorithm. arXiv:1712.01815, 2017.

[10] **Monroe, D. et al.** Chessformer: A Unified Architecture for Chess Modeling. ICLR, 2026.

Software: Leela Chess Zero (lc0 v0.33, onnx-trt backend), NVIDIA TensorRT and ModelOpt, ONNX Runtime, Syzygy tablebases.